

Primal-Dual iLQR for GPU-Accelerated Learning and Control in Legged Robots

Lorenzo Amatucci^{*,1,2}, João Sousa-Pinto^{*,3}, Giulio Turrisi¹, Dominique Orban⁴,
 Victor Barasuol¹, Claudio Semini¹

Abstract—This paper introduces a novel Model Predictive Control (MPC) implementation for legged robot locomotion that leverages GPU parallelization. Our approach enables both temporal and state-space parallelization by incorporating a parallel associative scan to solve the primal-dual Karush-Kuhn-Tucker (KKT) system. In this way, the optimal control problem is solved in $\mathcal{O}(\log^2(n) \log N + \log^2(m))$ complexity, instead of $\mathcal{O}(N(n+m)^3)$, where n , m , and N are the dimension of the system state, control vector, and the length of the prediction horizon. We demonstrate the advantages of this implementation over two state-of-the-art solvers (acados and crocoddyl), achieving up to a 60% improvement in runtime for Whole Body Dynamics (WB)-MPC and a 700% improvement for Single Rigid Body Dynamics (SRBD)-MPC when varying the prediction horizon length. The presented formulation scales efficiently with the problem state dimensions as well, enabling the definition of a centralized controller for up to 16 legged robots that can be computed in less than 25 ms. Furthermore, thanks to the JAX implementation, the solver supports large-scale parallelization across multiple environments, allowing the possibility of performing learning with the MPC in the loop directly in GPU. The code associated with this work can be found at <https://github.com/iit-DLSLab/mpx>.

Index Terms—Optimization and Optimal Control; Legged Robots; Multi-Contact Whole-Body Motion Planning and Control

I. INTRODUCTION

AMONG the many well-known control approaches available, Model Predictive Control (MPC) has proven to be highly effective in generating and controlling complex dynamic behaviors in robotic systems, especially legged robots, as shown by [1] and [2]. At the core of an MPC is the transcription of a task we want the robot to perform into an Optimal Control Problem (OCP) and the ability to solve it fast enough to be used in a closed-loop controller.

One of the most widely used methods in robotics for solving such OCP is Differential Dynamic Programming (DDP), which gained renewed attention through the work of Todorov et al. [3], who introduced a variant known as Iterative Linear Quadratic Regulator (iLQR). iLQR discards the second-order terms of the dynamics in the Hessian, sacrificing the local quadratic convergence of DDP in favor of faster update rates. However, a key limitation of this method is its single-shooting nature, which requires a feasible initial guess as a starting

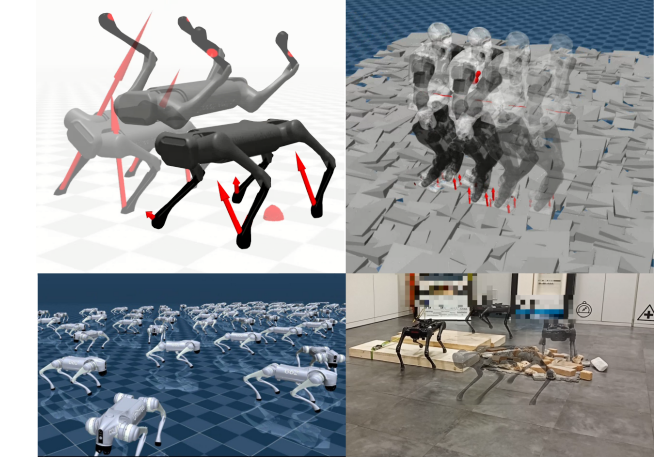


Fig. 1: Top left: the quadruped robot Aliengo performing a barrel roll. Top right: the humanoid robot Talos walking blindly on uneven terrain. Bottom left: an example of the massive parallelization achievable with our JAX formulation. Bottom right: Aliengo robot walking blindly on uneven terrain in a lab experiment.

point and often exhibits critical numerical issues. [4] overcame such limitation presenting a multiple shooting variant of the iLQR algorithm. More recently, [5] proposed a feasibility-driven multiple shooting approach to DDP with notable results also on real hardware [6]. Another common approach to the solution of the OCP is Sequential quadratic Programming (SQP). Compared to DDP-like algorithms, SQP has been more extensively developed for general-purpose solvers due to its flexibility in handling a broader range of constraints and objectives, as well as its robustness in dealing with infeasible iterations. Solvers like OCS2 [7] and acados [8] also showed relevant results on real hardware as demonstrated in [9] and [10], thanks to the tailoring of the solver to the structure of OCP. Recent work like [11] highlights the connection between SQP and DDP, describing how the sparsity structure of the OCP can be exploited not only in DDP-like solvers but also in other solver types, such as SQP or interior point-based solvers.

While the above methods have been shown to exploit the structure of the OCP, they have struggled to exploit new hardware accelerators like GPUs. Compared to CPUs, graphics cards can massively parallelize computation and are specifically designed for high-throughput linear algebra operations. Jeon et al. [12] tried to bridge the gap by developing Cusadi, a tool to convert expressions written in the well-known framework CasADI [13] to GPU. Cusadi only translates closed-form expressions into GPU-compatible code, lacking the branching and looping capabilities necessary for most solvers. Bishop et al., [14] introduced a GPU

^{*} Equal Contribution

¹ Dynamic Legged Systems Laboratory, Istituto Italiano di Tecnologia (IIT), Genova, Italy. E-mail: name.lastname@iit.it

² Dipartimento di Informatica, Bioingegneria, Robotica e Ingegneria dei Sistemi (DIBRIS), Università di Genova, Genova, Italy.

³ This work was done while João Sousa-Pinto was at Apple.

⁴ GERAD and Department of Mathematics and Industrial Engineering, Polytechnique Montréal, Montreal, Canada

accelerated Quadratic Programming (QP) solver, Relu-QP, leveraging the Alternating Direction Method of Multipliers (ADMM) algorithm, achieving performance superior to state-of-the-art CPU-based solutions. However, its applicability to Nonlinear Programming (NLP) is constrained by the need to pre-compute part of the algorithm offline. Lee et al. [15] implements an iLQR controller on GPU but only partially leverages its capabilities, limiting parallelization to the line search and gradient computation. In contrast, [16] focuses only on batching, solving multiple OCPs in parallel without exploiting state or temporal parallelism. [17] extends the approach of [15] by incorporating the temporal parallelization strategy introduced in [1]. However, [1] requires introducing approximations in the backward pass and performing a consensus sweep to maintain consistency. Frasch et al. [18], proposed a method to iteratively solve linear quadratic control exploiting parallel computation, while Wright et al. [19] showed how to partition dynamic programming for parallel computation. However, none of the mentioned methods achieved at the same time logarithmic time complexity and an exact formulation. On the other hand, Särkkä et al. [20] showed how parallel associative scan operations can be utilized to improve the computational complexity of LQR. When parallelized on a GPU, associative scans can compute the optimal control policy in $\mathcal{O}(\log^2(n) \log N + \log^2(m))$ instead of the $\mathcal{O}(N(n+m)^3)$ of the classical Riccati Recursion, where n and m are the dimensions of the system state and control input, and N is the prediction horizon length.

In this work, we designed an SQP method that uses an associative scan-based LQR solver to solve the primal-dual Karush-Kuhn-Tucker (KKT) system efficiently. The solver avoids the offline precalculation necessary for Relu-QP [14]. Our method benefits from a multiple shooting implementation, instead of the single shooting approach presented in [17]. Our algorithm fully exploits the parallelization capabilities of GPUs in the temporal, state, and control dimensions, while maintaining, in contrast to [17], an exact backward pass. We tailored the formulation for the deployment as a receding horizon controller for legged robot locomotion and analyzed its impact on learning and control.

Control: At first, the reduction in the computational complexity of the solver may appear to have limited significance in a practical application. Some of the already described methods, like OCS2 [7] and crocoddyl [5], have been successfully used for whole-body MPC even for experiments, as shown in [21] and [2]. To ensure that such an MPC formulation can run online on real hardware, the prediction horizon length and the number of robots that can be controlled simultaneously are constrained by the time complexity of the underlying algorithm. In particular, for solvers like OCS2, acados, and crocoddyl, the complexity scales linearly with the horizon length and cubically with the state dimension. In contrast, our approach scales with the square-log of the horizon length and logarithmically with the state and control dimensions. This improved scaling enables the use of more nodes at the same update rate, allowing for finer integration steps for a better approximation of the robot dynamic or, more critically, the inclusion of a Moving Horizon Estimator (MHE), such as

those presented by [22] or [23], within the same optimization loop, opening the pace for "end-to-end" MPCs capable of adapting and reacting to system changes directly using the sensor data. The reduction in complexity with respect to the state dimension enables better scalability across different robot morphologies. While other methods, such as the one presented by [24], use an approximate approach in the form of consensus ADMM to parallelize over the system state, our implementation leverages the inherent parallelism of matrix operations provided by the GPU. State space scaling becomes crucial when a centralized controller for collaborative tasks is considered. As demonstrated by [25] and [26], centralized controllers can handle complex tasks involving multiple collaborating agents. However, while these approaches achieve notable performances, the complexity of the task and the number of agents involved are constrained by their capability to solve the increasingly large OCPs at a reasonable control frequency. Our framework overcomes such barriers, as shown in Sec. IV-C.

Learning: In recent years, Reinforcement Learning (RL) has demonstrated remarkable performance, enhancing the robustness and capabilities of legged robots. RL controllers have surpassed model-based controllers in robustness against model mismatches and sensor noise. Cheng et al. [27] also demonstrated agile movements involving jumps on unstructured terrains with a real robot. However, RL policies have shown limitations in scenarios that involve crossing gaps and stepping stones, since the learning process efficiency is significantly affected due to the sparse reward signals typical of such tasks. Giftthaler et al. [28] proposed a solution to bridge the gap between learning-based and model-based controllers by incorporating MPC as a bias for the RL-policy. In contrast, works such as [29] and [30] integrated learned behaviors directly into the optimization process. Despite the noticeable results, all of the aforementioned methods that mix model-based controllers and RL suffered from a slower training process due to the dependency on CPU-based solvers. In contrast, our implementation is entirely developed in JAX [31], and it can be coupled with GPU-based simulators such as IsaacLab [32] or Mujoco XLA [33] to achieve a significant speed-up in simulation throughput.

A. Contribution

To summarize, the major contributions of this work are:

- a novel GPU-accelerated MPC for legged robots that achieves logarithmic scaling in computation complexity on the horizon length, squared-logarithmic scaling in the state and control dimensions, and can easily be parallelized for use with data-driven approaches;
- a detailed analysis of the performance benefits and limitations of the proposed algorithm with respect to state-of-the-art solvers;
- an open-source code repository for the rapid prototyping of MPC for legged robots and loco-manipulation, written in JAX; the code provides tools for controlling legged robots and also supports large-scale parallelization.

B. Outline

This paper is organized as follows. Sec. II describes the details of the proposed solver. Sec. III presents the dynamical models used in our formulation. Sec. IV shows the benefits of our approach on comparative simulations against state-of-the-art solutions. Finally, Sec. V draws the final conclusions.

II. PRIMAL-DUAL LQR

A. Optimal Control Problem

We start describing the OCP at the base of the MPC formulation we are presenting. In a OCP, we transcribe not only the task we want to be performed but also the physical limits the system needs to respect. In this paper, we consider OCPs of the following form:

$$\min_{\mathbf{x}, \mathbf{u}} \quad l_{N+1}(\mathbf{x}_{N+1}) + \sum_{i=0}^N l_i(\mathbf{x}_i, \mathbf{u}_i) \quad (1a)$$

$$s.t. \quad \mathbf{x}_{i+1} = \mathbf{h}_i(\mathbf{x}_i, \mathbf{u}_i) \quad (1b)$$

$$\mathbf{u}_i \in \mathbf{U}_i \quad (1c)$$

$$\mathbf{x}_i \in \mathbf{X}_i \quad (1d)$$

$$\mathbf{x}_0 = \hat{\mathbf{x}}_0 \quad (1e)$$

where $\mathbf{x}_i$ is the system state and $\mathbf{u}_i$ is the control input. $l(\mathbf{x}_i, \mathbf{u}_i)$ is the stage cost made of a quadratic tracking and regularization terms, while $l(\mathbf{x}_N)$ is the terminal cost, and $\mathbf{h}(\mathbf{x}_i, \mathbf{u}_i)$ is the system dynamics. Finally, $\mathbf{X}_i$ and $\mathbf{U}_i$ are, respectively, the set of feasible system states and control inputs. The remainder of this section will describe the methods we use to solve the optimization problem posed by the OCP formulation presented in (1). By leveraging both the knowledge of the problem structure and exploiting modern hardware accelerators, we are able to obtain logarithmic time complexity in horizon length N as well as square-log for the state x and input u dimensions. For simplicity, in this section, we avoid treating the inequality constraint in (1d) and (1c). These details will be treated separately in Sec. II-F. We now focus on the solution of the equality-constrained optimization problem, which only presents the initial condition (1e) and dynamics (1b) as constraints. In particular, we are going to derive a multiple-shooting approach, which, in contrast to the single-shooting one, keeps both the state and control as optimization variables. We first derive an efficient SQP algorithm that exploits the Riccati recursion to solve the equality constrained QP Sec. II-C, and then we focus on the temporal parallelization of the solver through the use of parallel associative scan, Sec. II-D.

B. Sequential Quadratic Programming

We start by defining the Lagrangian of problem in (1) as

$$\mathcal{L}(\mathbf{x}, \mathbf{u}, \boldsymbol{\lambda}) = l_{N+1}(\mathbf{x}_{N+1}) + \boldsymbol{\lambda}_0^T (\hat{\mathbf{x}}_0 - \mathbf{x}_0) + \sum_{i=0}^N l_i(\mathbf{x}_i, \mathbf{u}_i) + \boldsymbol{\lambda}_{i+1}^T (\mathbf{h}_i(\mathbf{x}_i, \mathbf{u}_i) - \mathbf{x}_{i+1}), \quad (2)$$

where $\boldsymbol{\lambda}_i$ for $i = 1 \dots N+1$ are the Lagrangian multipliers associated with the dynamics constraints (also known as

costates). In an SQP method, we iteratively solve the QP derived from the quadratic approximation of the Lagrangian and the linearization of the constraints at the current guess, $\mathbf{x}^k, \mathbf{u}^k$, where k represents the last iterate. The QP is then written as

$$\begin{aligned} \min_{\delta \mathbf{x}, \delta \mathbf{u}} \quad & \sum_{i=1}^N \begin{bmatrix} \mathbf{q}_i \\ \mathbf{r}_i \end{bmatrix}^\top \begin{bmatrix} \delta \mathbf{x}_i \\ \delta \mathbf{u}_i \end{bmatrix} + \frac{1}{2} \begin{bmatrix} \delta \mathbf{x}_i \\ \delta \mathbf{u}_i \end{bmatrix}^\top \begin{bmatrix} \mathbf{Q}_i & \mathbf{S}_i^\top \\ \mathbf{S}_i & \mathbf{R}_i \end{bmatrix} \begin{bmatrix} \delta \mathbf{x}_i \\ \delta \mathbf{u}_i \end{bmatrix} \\ & + \mathbf{p}_{N+1}^\top \delta \mathbf{x}_{N+1} + \frac{1}{2} \delta \mathbf{x}_{N+1}^\top \mathbf{P}_{N+1} \delta \mathbf{x}_{N+1} \\ & \delta \mathbf{x}_0 = \hat{\mathbf{x}}_0 - \mathbf{x}_0^k \\ & \delta \mathbf{x}_{i+1} = \mathbf{A}_i \delta \mathbf{x}_i + \mathbf{B}_i \delta \mathbf{u}_i + \mathbf{b}_i \quad \text{for } i = 0, \dots, N, \end{aligned} \quad (3)$$

where the search directions $\delta \mathbf{x}$ and $\delta \mathbf{u}$ are defined as

$$\begin{aligned} \delta \mathbf{x}_i &= \mathbf{x}_i^{k+1} - \mathbf{x}_i^k \quad \text{for } i = 0, \dots, N+1 \\ \delta \mathbf{u}_i &= \mathbf{u}_i^{k+1} - \mathbf{u}_i^k \quad \text{for } i = 0, \dots, N. \end{aligned}$$

The linearized dynamics is defined as

$$\mathbf{A}_i = \frac{\partial \mathbf{h}_i}{\partial \mathbf{x}_i}, \mathbf{B}_i = \frac{\partial \mathbf{h}_i}{\partial \mathbf{u}_i}, \mathbf{b}_i = \mathbf{h}_i - \mathbf{x}_{i+1}^k.$$

The linear terms in the objective function are defined as

$$\mathbf{q}_i = \nabla_{\mathbf{x}_i} \mathcal{L}, \quad \mathbf{r}_i = \nabla_{\mathbf{u}_i} \mathcal{L}, \quad \mathbf{p}_{N+1} = \nabla_{\mathbf{x}_{N+1}} \mathcal{L}.$$

Finally, the Quadratic terms are defined as

$$\mathbf{P}_{N+1} = \nabla^2 \mathcal{L}_{N+1}, \quad \begin{bmatrix} \mathbf{Q}_i & \mathbf{S}_i^\top \\ \mathbf{S}_i & \mathbf{R}_i \end{bmatrix} = \nabla^2 \mathcal{L}_i,$$

where we removed the dependency of $\mathbf{x}_i^k$ and $\mathbf{u}_i^k$ from the dynamics $\mathbf{h}_i$ and the cost function l_i .

C. Primal & dual problem solution

As shown by [34], such QPs can be efficiently solved by exploiting the problem structure. By applying the Riccati Recursion, starting from the last (i.e. with index N) node in the horizon and going backward in time, we obtain:

$$\begin{aligned} \mathbf{G}_i &= \mathbf{R}_i + \mathbf{B}_i^\top \mathbf{P}_{i+1} \mathbf{B}_i, \\ \mathbf{H}_i &= \mathbf{S}_i + \mathbf{B}_i^\top \mathbf{P}_{i+1} \mathbf{A}_i, \\ \mathbf{h}_i &= \mathbf{B}_i^\top (\mathbf{p}_{i+1} + \mathbf{P}_{i+1} \mathbf{b}_i) + \mathbf{r}_i, \\ \mathbf{K}_i &= -\mathbf{G}_i^{-1} \mathbf{H}_i, \\ \mathbf{k}_i &= -\mathbf{G}_i^{-1} \mathbf{h}_i, \\ \mathbf{P}_i &= \mathbf{Q}_i + \mathbf{A}_i^\top \mathbf{P}_{i+1} \mathbf{A}_i + \mathbf{K}_i^\top \mathbf{H}_i, \\ \mathbf{p}_i &= \mathbf{q}_i + \mathbf{A}_i^\top (\mathbf{p}_{i+1} + \mathbf{P}_{i+1} \mathbf{b}_i) + \mathbf{K}_i^\top \mathbf{h}_i. \end{aligned} \quad (4)$$

where $\mathbf{K}_i$ and $\mathbf{k}_i$ are, respectively, the feedback and feed-forward term of the optimal control policy. We can now retrieve the control and state vectors by recursively applying the forward pass starting from the initial condition as follows:

$$\delta \mathbf{u}_i = \mathbf{K}_i \delta \mathbf{x}_i + \mathbf{k}_i, \quad \delta \mathbf{x}_{i+1} = \mathbf{A}_i \delta \mathbf{x}_i + \mathbf{B}_i \delta \mathbf{u}_i + \mathbf{b}_i. \quad (5)$$

Once we retrieve the values of $\mathbf{P}_i$ and $\mathbf{p}_i$ for $i = 0, \dots, N+1$, we can compute the update of the Lagrangian multiplier $\delta \boldsymbol{\lambda}$ in parallel. This is achieved by applying, at each node, the following update:

$$\delta \boldsymbol{\lambda}_i = \mathbf{P}_i \delta \mathbf{x}_i + \mathbf{p}_i. \quad (6)$$

D. Parallel Associative Scan

In this section, we will briefly recap how parallel associative scans can be used to solve the primal problem as shown for the first time by [20]. Given a sequence of elements $a_1, \dots, a_N$ and an associative operator $\otimes$ defined on them, a parallel associative scan can be used to evaluate the values $s_1, \dots, s_N$ as:

$$(s_1, s_2, \dots, s_N) = (a_1, a_1 \otimes a_2, \dots, a_1 \otimes a_2 \otimes \dots \otimes a_N) \quad (7)$$

in $\mathcal{O}(\log N)$ as shown by [35]. This is possible since $\otimes$ is an associative operation, allowing the computation to be reorganized into smaller interdependent subproblems. To make use of this property, we first define the Conditional Value Function (CVF) $V_{i \rightarrow j}(\mathbf{x}_i, \mathbf{x}_j)$ as the minimal cost related to the optimal trajectory that goes from state $\mathbf{x}_i$ at node i in the horizon to state $\mathbf{x}_j$ at node j , with $i < j$. Given the connection between the value function of (3) and its CVF, $V_i(\mathbf{x}_i) = \max_{\mathbf{x}_{N+1}} V_{i \rightarrow N+1}(\mathbf{x}_i, \mathbf{x}_{N+1})$, we will use the scan to evaluate $V_i(\mathbf{x}_i)$ for $i = 0, \dots, N+1$. We begin by initializing the elements a_i with the CVF $V_{i \rightarrow i+1}$ defined between two adjacent states $\mathbf{x}_i$ and $\mathbf{x}_{i+1}$. Next, we define the operator $\otimes$ for dynamic programming as:

$$V_{i \rightarrow k} \otimes V_{k \rightarrow j} = \min_{\mathbf{x}_k} \{V_{i \rightarrow k} + V_{k \rightarrow j}\} \quad (8)$$

where $\mathbf{x}_k$ is a third intermediate state between i and j . Note that this operator is associative since the $\min$ operator is associative. We can now perform an associative scan on the CVF to obtain $V_{i \rightarrow N+1}$ in a single pass, thus $V_i(\mathbf{x}_i)$ for all i in the horizon. To derive the combination rule for (8), we start by noting that the CVF of (3) is also a quadratic program with affine equality constraint (1b), which can be represented in its dual form [36] by introducing the Lagrangian multiplier η :

$$V_{i \rightarrow j}(\mathbf{x}_i, \mathbf{x}_j) = \max_{\eta} \frac{1}{2} \mathbf{x}_i^\top \tilde{\mathbf{P}}_{i,j} \mathbf{x}_i + \tilde{\mathbf{p}}_{i,j}^\top \mathbf{x}_i - \frac{1}{2} \eta^\top \tilde{\mathbf{C}}_{i,j} \eta - \eta^\top (\mathbf{x}_j - \tilde{\mathbf{A}}_{i,j} \mathbf{x}_i - \tilde{\mathbf{b}}_{i,j}) \quad (9)$$

where the tilde $\tilde{\cdot}$ is used to distinguish the values related to the conditional value function from the one in (4), while the subscripts are used to specify the initial and final state considered. It can be proved that, given $V_{i \rightarrow k}$ and $V_{k \rightarrow j}$ in the form of (9), $V_{i \rightarrow j}$ will still have the form of (9) and is characterized by the combination rule

$$\begin{aligned} \tilde{\mathbf{P}}_{i,j} &= \tilde{\mathbf{A}}_{i,k}^\top \left(\mathbf{I} + \tilde{\mathbf{P}}_{k,j} \tilde{\mathbf{C}}_{i,k} \right)^{-1} \tilde{\mathbf{P}}_{k,j} \tilde{\mathbf{A}}_{i,k} + \tilde{\mathbf{P}}_{i,k}, \\ \tilde{\mathbf{p}}_{i,j} &= \tilde{\mathbf{A}}_{i,k}^\top \left(\mathbf{I} + \tilde{\mathbf{P}}_{k,j} \tilde{\mathbf{C}}_{i,k} \right)^{-1} (\tilde{\mathbf{p}}_{k,j} - \tilde{\mathbf{P}}_{k,j} \mathbf{b}_{i,k}) + \tilde{\mathbf{p}}_{i,k}, \\ \tilde{\mathbf{A}}_{i,j} &= \tilde{\mathbf{A}}_{k,j} \left(\mathbf{I} + \tilde{\mathbf{C}}_{i,k} \tilde{\mathbf{P}}_{k,j} \right)^{-1} \tilde{\mathbf{A}}_{i,k}, \\ \tilde{\mathbf{C}}_{i,j} &= \tilde{\mathbf{A}}_{k,j} \left(\mathbf{I} + \tilde{\mathbf{C}}_{i,k} \tilde{\mathbf{P}}_{k,j} \right)^{-1} \tilde{\mathbf{C}}_{i,k} \tilde{\mathbf{A}}_{k,j}^\top + \tilde{\mathbf{C}}_{k,j}, \\ \tilde{\mathbf{b}}_{i,j} &= \tilde{\mathbf{A}}_{k,j} \left(\mathbf{I} + \tilde{\mathbf{C}}_{i,k} \tilde{\mathbf{P}}_{k,j} \right)^{-1} (\tilde{\mathbf{b}}_{i,k} - \tilde{\mathbf{C}}_{i,k} \mathbf{p}_{k,j}) + \tilde{\mathbf{b}}_{j,k}. \end{aligned} \quad (10)$$

We now initialize the values a_i of a reverse associative scan, using the conditional value function $V_{i \rightarrow i+1}$ characterized by

$$\begin{aligned} \tilde{\mathbf{A}}_{i,i+1} &= \mathbf{A}_i - \mathbf{B}_i \mathbf{R}_i^{-1} \mathbf{S}_i, & \tilde{\mathbf{P}}_{i,i+1} &= \mathbf{Q}_i - \mathbf{S}_i^\top \mathbf{R}_i^{-1} \mathbf{S}_i, \\ \tilde{\mathbf{C}}_{i,i+1} &= \mathbf{B}_i \mathbf{R}_i^{-1} \mathbf{B}_i^\top, & \tilde{\mathbf{p}}_{i,i+1} &= \mathbf{q}_i - \mathbf{S}_i^\top \mathbf{R}_i^{-1} \mathbf{r}_i, \\ \tilde{\mathbf{b}}_{i,i+1} &= \mathbf{b}_i - \mathbf{B}_i \mathbf{R}_i^{-1} \mathbf{r}_i, \end{aligned} \quad (11)$$

and, in the case of $V_{N,N+1}$, by

$$\begin{aligned} \tilde{\mathbf{P}}_{N,N+1} &= \mathbf{Q}_{N+1}, & \tilde{\mathbf{A}}_{N,N+1} &= 0, & \tilde{\mathbf{b}}_{N,N+1} &= 0, \\ \tilde{\mathbf{p}}_{N,N+1} &= \mathbf{q}_{N+1}, & \tilde{\mathbf{C}}_{N,N+1} &= 0. \end{aligned} \quad (12)$$

From the solution of the parallel associative scan using the combination rule in (10), we obtain $V_{i \rightarrow N+1}(\mathbf{x}_i, \mathbf{x}_{N+1})$ for $i = 0, \dots, N+1$. Finally, recalling that $\max_{\mathbf{x}_{N+1}} V_{i \rightarrow N+1}(\mathbf{x}_i, \mathbf{x}_{N+1})$ is equal to the value function $V_i(\mathbf{x}_i)$ of the problem in (3) and that the value function is also in the same form of (9), we get that $\mathbf{P}_i = \tilde{\mathbf{P}}_{i,N+1}$ and $\tilde{\mathbf{p}}_i = \mathbf{p}_{i,N+1}$. This means that we obtained all $\mathbf{P}_i$ and $\mathbf{p}_i$ for $i = 0, \dots, N+1$ that can then be used in (II-C) to calculate $\mathbf{K}$ and $\mathbf{k}$ in one pass in parallel. Thanks to a GPU-accelerated scan implementation, we can obtain the control policy over the entire horizon with an overall computational complexity of $\mathcal{O}(\log^2(n) \log N + \log^2(m))$. This follows from the fact that both computing and initializing the scan involve a matrix inversion, which can be performed in parallel with complexity $\mathcal{O}(\log^2(n))$ [37]. The parallel associative scan can also be used to retrieve the optimal trajectory in $\mathcal{O}(\log n \log N + \log m)$. We start by plugging into the linearized dynamics the optimal control law we evaluated, obtaining

$$\delta \mathbf{x}_{i+1} = \bar{\mathbf{A}}_i \delta \mathbf{x}_i + \bar{\mathbf{b}}_i, \quad (13)$$

where $\bar{\mathbf{A}}_i = \mathbf{A}_i + \mathbf{B}_i \mathbf{K}_i$ and $\bar{\mathbf{b}}_i = \mathbf{B}_i \mathbf{k}_i + \mathbf{b}_i$. We can define the conditional optimal trajectory as follows:

$$\bar{\mathbf{h}}_{i \rightarrow j}(\delta \mathbf{x}_i, \delta \mathbf{x}_j) = \bar{\mathbf{A}}_{i,j} \delta \mathbf{x}_i + \bar{\mathbf{b}}_{i,j}. \quad (14)$$

[20] showed that from the combination of two conditional optimal trajectory $\mathbf{h}_{i \rightarrow k}$ and $\mathbf{h}_{k \rightarrow j}$ we obtain $\mathbf{h}_{i \rightarrow j}$ that can be written in the same form as (14) with

$$\bar{\mathbf{A}}_{i,j} = \bar{\mathbf{A}}_{i,k} \bar{\mathbf{A}}_{k,j} \quad \bar{\mathbf{b}}_{i,j} = \bar{\mathbf{A}}_{k,j} \bar{\mathbf{b}}_{i,k} + \bar{\mathbf{b}}_{k,j}. \quad (15)$$

We can initialize the element of parallel scan $a_i = \mathbf{h}_{i \rightarrow i+1}(\mathbf{x}_i)$ as

$$\bar{\mathbf{A}}_{i,i+1} = \bar{\mathbf{A}}_i, \quad \bar{\mathbf{b}}_{i,i+1} = \bar{\mathbf{b}}_i, \quad (16)$$

for $i = 1, \dots, N$, while for $i = 0$ we set $\bar{\mathbf{A}}_{0,1} = \mathbf{0}$ and $\bar{\mathbf{b}}_{0,1} = \bar{\mathbf{A}}_0 \delta \mathbf{x}_0 + \bar{\mathbf{b}}_0$. From the parallel scan, we obtained the updated optimal state as:

$$\delta \mathbf{x}_i = a_0 \otimes a_1 \otimes \dots \otimes a_{i-1} \quad (17)$$

E. Parallel Line Search

Once we update the optimal directions $\delta \mathbf{x}$, $\delta \mathbf{u}$, and $\delta \lambda$, we can perform a backtracking line search to calculate the step length α to finally get the new guess as

$$\begin{aligned} \mathbf{x}^{k+1} &= \mathbf{x}^k + \alpha \delta \mathbf{x}, & \lambda^{k+1} &= \lambda^k + \alpha \delta \lambda, \\ \mathbf{u}^{k+1} &= \mathbf{u}^k + \alpha \delta \mathbf{u}. \end{aligned} \quad (18)$$

The SQP linear rollout allows for the computation to be easily performed in parallel for the horizon. To evaluate α , we implemented a filter line search similar to the one implemented in [38]. The filter line search ensures that the step taken in the iteration reduces the constraint violation or the cost. Such a method has already been successfully deployed for legged robot control in [2]. We measure the satisfaction of the dynamics constraints as

$$\theta = \sum_{i=0}^{N+1} \|x_{i+1} - h(x_i, u_i)\|_2. \quad (19)$$

As for [2], when the constraint violation θ is higher than a threshold, we reject the step if it further increases θ . If the violation is below the threshold and the current step is a descent direction, then we apply an Armijo condition [39]. On the other hand, if the current iterate is not in a descent direction, we require that at least the cost or θ is decreased to accept the step. To leverage GPU parallelism, we evaluate the line-search in parallel (at a fixed grid of ten step sizes, i.e. $\alpha \in \{2^0, \dots, 2^{-9}\}$) and select the largest α that satisfies one of the line-search conditions.

F. Practical Implementation

As it is commonly done in practice, we also perform a Gauss-Newton approximation, ignoring the second-order term of the dynamics in (3). We can further consider that in our case, the cost for the MPC can be expressed as a non-linear least squares problem in the form $\frac{1}{2} \sum_{i=0}^N \|\epsilon(x, u)\|_W$, where W is a weight matrix. The Hessian approximation reduces to:

$$\begin{aligned} Q_{N+1} &= \nabla_x \epsilon_{N+1}^\top W_{N+1} \nabla_x \epsilon_{N+1} & S_i &= \nabla_x \epsilon_i^\top W_i \nabla_u \epsilon_i \\ Q_i &= \nabla_x \epsilon_i^\top W_i \nabla_x \epsilon_i & R_i &= \nabla_u \epsilon_i^\top W_i \nabla_u \epsilon_i \end{aligned} \quad (20)$$

Such an approximation also guarantees that 3 is convex. As described in Sec. II-A, the inequality constraints in (1c) and (1d) were not explicitly included in the solver formulation. However, these constraints are critical for MPC as they represent the physical limitations of the system, such as friction cones or joint limits. To account for these limitations in the optimization, we incorporated them as a relaxed barrier function:

$$\mathcal{B}(\xi, \mu, \delta) \begin{cases} -\mu \ln(\xi), & \text{if } \xi \geq \delta, \\ \frac{\mu}{2} \left(\left(\frac{\xi - 2\delta}{\delta} \right)^2 - 1 \right) - \mu \ln(\delta), & \text{if } \xi < \delta, \end{cases} \quad (21)$$

where $\xi(x, u) < 0$ is the considered constraint, and μ and δ are parameters tuned for each constraint. Following the example of [2], to maintain the convexity of 3 the Hessian approximation of the constraints is constructed as

$$\begin{aligned} Q_{N+1} &= \nabla_x \epsilon_{N+1}^\top W_{N+1} \nabla_x \epsilon_{N+1} + \nabla_x \xi_{N+1}^\top \nabla_{\xi\xi} \mathcal{B}_{N+1} \nabla_x \xi_{N+1} \\ Q_i &= \nabla_x \epsilon_i^\top W_i \nabla_x \epsilon_i + \nabla_x \xi_i^\top \nabla_{\xi\xi} \mathcal{B}_i \nabla_x \xi_i \\ S_i &= \nabla_x \epsilon_i^\top W_i \nabla_x \epsilon_i + \nabla_x \xi_i^\top \nabla_{\xi\xi} \mathcal{B}_i \nabla_u \xi_i \\ R_i &= \nabla_u \epsilon_i^\top W_i \nabla_u \epsilon_i + \nabla_u \xi_i^\top \nabla_{\xi\xi} \mathcal{B}_i \nabla_u \xi_i \end{aligned} \quad (22)$$

As a common practice, while using the MPC in closed-loop with the robot, we perform one iteration only of the algorithm described in Sec. II-A per control loop. To increase

the robustness of the method, we warm-start each call of the solver with the previous prediction just shifted by one time-step as shown by [40].

Finally, to guarantee the differentiability of the solver through JAX-based automatic differentiation, we avoided the use of *while* loop in the code. Especially in line-search, we evaluate a fixed number of alpha values in parallel.

III. MODEL PREDICTIVE CONTROL

In this section, we describe briefly the dynamics model $h(x_i, u_i)$ presented in (1b), introducing two different models used for the formulation of the OCP presented in the result Sec. IV. The efficacy of the control input obtained from a MPC strategy is directly related to the rate at which such a control strategy can be re-planned and to the accuracy of its predictions, as shown by [41]. Therefore, we consider two main formulations from the literature: one based on the Single Rigid Body Dynamics (SRBD) model and one based on the Whole Body (WB) model. The first approximates the system as a single body with constant inertia, discarding the limb joint positions and the inertial variation from different limb configurations. This model is particularly effective in quadruped robots, where the limb mass is, in most cases, designed to be less than 10% of the total mass [42]. On the other hand, the latter completely captures the dynamics of the robot, enabling the maximum exploitation of the system's capabilities. This benefit comes at the cost of handling a larger state space and highly non-convex models, thus affecting the maximum frequency at which the optimization problem can be solved. Moreover, the SRBD-MPC state is limited to the center of mass linear and angular positions and velocities. This means that, when representing the robot's orientation using quaternions, the state lies in $\mathbb{R}^7$. For such model, the control input consists only of the Ground Reaction Forces (GRF), and thus lies in $\mathbb{R}^{3n_c}$, where n_c is the number of contact points, meaning that, for the evaluations with the quadruped, in Sec. IV n is in $\mathbb{R}^{13}$ and m in $\mathbb{R}^{12}$.

In contrast, the WB-MPC also includes joint positions and velocities in the state, and joint torques in the control input. This increases the state dimensionality to $\mathbb{R}^{13+2n_{\text{joint}}}$ and the control input dimensionality to $\mathbb{R}^{3n_c+n_{\text{joint}}}$, that translate for the quadruped model in Sec. IV in n in $\mathbb{R}^{37}$ and m in $\mathbb{R}^{24}$.

Beyond the difference in dimensionality, the WB model also introduces greater complexity. While the SRBD model features bilinear terms only in the angular dynamics, the WB model has highly nonlinear dependencies on the joint configuration. As a result, the overall OCP becomes significantly harder to solve.

IV. RESULTS

In this section, we present the results of various analyses that highlight the strength of the proposed approach. We evaluate the performance of our algorithm using the two models introduced in Sec. III (the SRBD and WB models). To benchmark our implementation against state-of-the-art solvers, we used acados [8] (SQP) for comparisons on the SRBD model and crocoddyl [5] (DDP) for the WB model. We used acados for the SRBD comparison, as it could be easily adapted

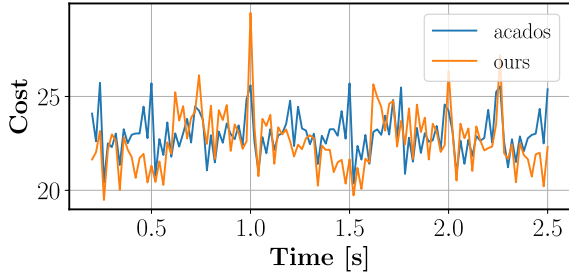


Fig. 2: Comparison of the MPC optimal cost, evaluated from the same initial condition with acados [8] (in blue) and with our implementation (in orange). Both solvers are evaluated at each call of the MPC and the costs are recorded.

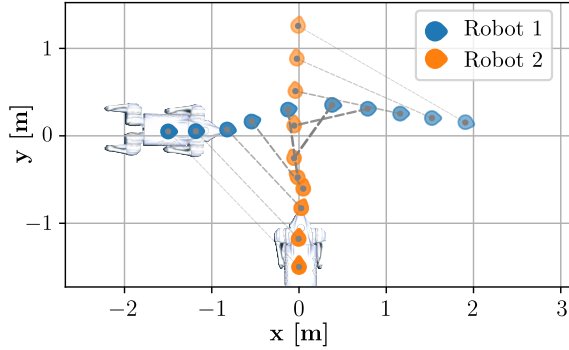


Fig. 3: Path of the two robots tracking their desired trajectories that will lead to a collision. We visualize the distance between the two robots with dashed gray lines of different shades, the darker the smaller distance.

for the evaluations with multiple robots, shown in Fig. 6 and 7. On the other hand, we used crocoddyl for the horizon length comparison with the WB model, as it showed significantly better performance than acados with the more complex model as shown in Fig. 4.

Our analysis includes investigating how the average solving time scales with the horizon length across all models and solvers. Additionally, we assess the solver’s performance as the system state dimension increases, as well as the vectorization capabilities of our approach. All the solving time comparisons consider a single iteration for each algorithm. The tests were conducted on a desktop computer equipped with an Intel Core i7-13700KF and an NVIDIA RTX 3080. We used Mujoco XLA, to evaluate the WB dynamics. In all the presented benchmarks, if not stated differently, we used the 15 kg Unitree Go2 robot, a torque-controlled quadruped platform, with a horizon length N of 50 nodes. As Fig. 1 and the accompanying video show, our WB-MPC can also be implemented for real-time control of other robot morphologies, like humanoids.

A. Performance evaluation

First, we evaluate the ability of our solver to generate complex dynamic maneuvers. In this scenario, we are generating a barrel roll motion with the Aliengo robot, as shown in Fig. 1. Thanks to its multiple shooting formulation, we can initialize the solver with an infeasible initial guess, and the solver converges in just 27 iterations to a solution that completes the task. We also assess the performance of the proposed solver in terms of solution optimality. In Fig. 2, we compared the

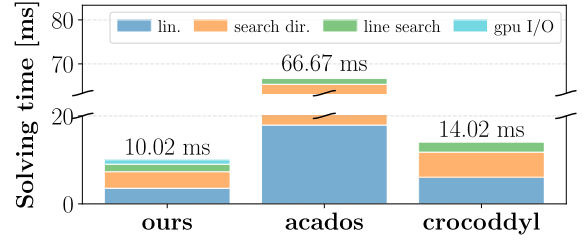


Fig. 4: Breakdown of the solving times for our implementation, acados (SQP) and crocoddyl (FDDP).

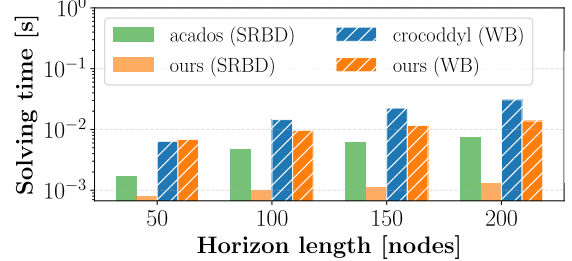


Fig. 5: Average solving times when varying the horizon length (number of nodes) for the SRBD model (top) and WB model (bottom). In blue and green, we plot the solution time of two different state-of-the-art solvers (respectively acados [8] and crocoddyl [5]), while in orange, we plot the one of our implementation.

values of the cost from the optimal solution given by our implementation against the one given by acados, which is chosen for this comparison because it allows using the same cost function as the one in our formulation. In this scenario, we are using the whole-body model and are controlling the robot in simulation at 50Hz. The robot is tasked to trot with a forward speed of 0.3 m/s while being randomly pushed by an external disturbance of 50N for 0.25s. At every control loop, we record the value of the cost functional from both solvers, comparing solutions that start from the same initial condition. Figure 2 reports the values recorded during a 2.5s time span. Although the two solutions are comparably similar, our implementation results in a 20% reduction in the average cost along the trajectory.

To finally validate the MPC, we also performed experiments on the real hardware, as shown in Fig. 1 and in the accompanying video. For the real experiments, the solver runs on an Nvidia RTX 4050 laptop GPU. Figure 1 shows snapshots of the robot walking blindly up the steps and through uneven terrain using only proprioceptive data for state estimation [43]. We also evaluated the proposed approach when used to control two different robots at the same time in a centralized control implementation in simulation. In this example, the two robots are controlled by the same MPC that evaluates the control input for both systems at 50Hz. The MPC also includes a collision avoidance constraint, in the form of a quadratic penalty term. In the tested scenario, the two robots are tasked to track two perpendicular trajectories that cross each other and would lead to a collision. Figure 3 shows the resulting path of the two robots, where the dashed grey lines represent the distance between the two robots. As shown, both robots deviate from their desired paths to avoid collisions, demonstrating coordinated behavior. While being just a simple example, this result highlights the potential of our MPC in solving collaborative tasks. Further implementation

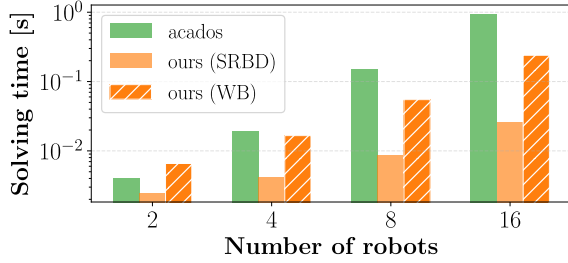


Fig. 6: Our implementation can control multiple robots in the same optimization, a typical scenario found in centralized controllers. In green, we depict the solution time of acados, while in orange, ours, considering the SRBD model. Furthermore, in dashed orange, we report the result obtained by our implementation using the WB model.

in more complex scenarios, such as collaborative carrying, is left for future work. Figure 4 presents the average solving time breakdown for the three solvers in the quadruped trotting scenario with a horizon of $N=100$. It highlights the sequential backward pass in *crocodyl*, the QP solver call in *acados*, and the parallel scan in our method for evaluating the search direction. A minor computational overhead of approximately $1ms$ arises from GPU communication, as only the initial state is sent and the commanded torques, joint positions, and velocities are retrieved.

B. Temporal Parallelization

In this subsection, we show the benefit of our implementation against the state-of-the-art solvers *acados* [8] and *crocodyl* [5], in terms of average solving time for increasing horizon length. For this assessment, we initialized the robot in a feasible random pose and joint configuration and tasked the controller to perform a forward trot at a speed of $0.5m/s$. *Acados* uses the same cost and dynamics formulation as in our controller, while for the comparison with *crocodyl* we adapted our reference generator and cost to match the reference used in the example provided with *crocodyl*'s libraries. As reported in Fig. 5, our implementation shows faster update rates than both solvers in almost all tested scenarios. For the assesment against *acados*, with the SRBD model, our implementation is always faster. For the case of the WB formulation, *crocodyl* is able to outperform our formulation with a prediction horizon smaller than 80 nodes. This last outcome is primarily attributed to overhead involved in transferring data to and from the GPU, which, for smaller problems, can be detrimental. Noticeably, our formulation can achieve an update rate of 50Hz for a horizon length of 200 nodes.

C. State Parallelization

In this subsection, we discuss the benefits of our implementation in the context of centralized controllers. In our tests, we evaluated the average solving times for the MPC while progressively increasing the number of robots included in the same optimization. This approach simulated a centralized controller managing n different robots. For the SRBD model, we compared our implementation against *acados*. For the WB model comparison, *crocodyl* was excluded since its model interface does not allow for a customization that is needed for

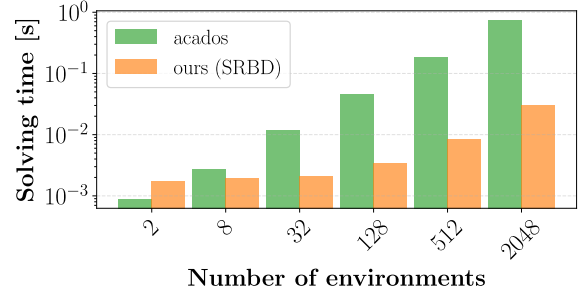


Fig. 7: Comparison of the solving time achieved by our implementation against the batch-parallelized solver feature of *acados*, when varying the number of environments being simulated in parallel. In this example, each environment has a dedicated MPC.

a fair evaluation. As Fig. 6 shows, our implementation outperformed *acados* when considering the SRBD model, achieving a computational time of 25 ms in the presence of 16 robots, enough for the real-time control of the systems. When using the WB model, instead, computation becomes significantly more demanding for the implementation with *acados* and we could not record data for the comparison. As shown in Fig. 6, our algorithm successfully controls up to four robots (WB model case) at 20 Hz. To the best of the authors' knowledge, this level of performance has never been achieved before.

	n_{env}	Control Frequency	Real-Time Factor
SRBD-MPC	4096	50 Hz	370×
SRBD-MPC	4096	25 Hz	570×
WB-MPC	4096	50 Hz	35×
WB-MPC	4096	25 Hz	75×

TABLE I: Performance metric

D. Model-Based Learning

In this subsection, we highlight the vectorization capabilities of our controller. Thanks to its full implementation in JAX, we can easily vectorize the computation of the MPC. Such ability is critical in the training scenario as shown in [28] or at runtime as shown in [44]. In Fig. 7, we compare our approach to the batched version of *acados*, which uses OpenMP to parallelize the computation of independent MPC instances on the CPU. As the plot shows, *acados* is competitive with the proposed MPC only for small batch sizes. This limitation comes from the reduced number of cores available on a CPU compared to the parallelization capabilities of modern GPUs.

Finally, Table I summarizes the performance of our MPC when batched across multiple environments with Mujoco XLA as the simulator. In this setup, we generate 4096 environments, each one running its own MPC. For the SRBD model, we control the robot at an update rate of 50 Hz and we achieve a value of 570 seconds of simulated time per second. Meanwhile, for the WB model, closing the loop at 25 Hz, we achieve a real-time factor of 75x.

V. CONCLUSION

This work presents a novel GPU-accelerated MPC framework implemented for legged robot locomotion. By leveraging the parallel processing capabilities of GPUs and implementing a Primal-Dual iLQR solver in JAX, we achieve logarithmic scaling in horizon length and square-log scaling with state

and control dimensions. Our approach demonstrates significant improvements over state-of-the-art solvers, achieving higher computational efficiency and scalability. This allows for the optimization of centralized controllers for multiple robots and the integration of large-scale parallel environments, enhancing learning-based control frameworks. Future works will include the exploitation of the presented MPC as a bias in the learning process of a locomotion policy. Furthermore, we would like to include a more sophisticated methodology for dealing with inequality constraints.

REFERENCES

- [1] F. Farshidian, E. Jelavic, A. Satapathy, M. Gifthalder, and J. Buchli, “Real-time motion planning of legged robots: A model predictive control approach,” in *IEEE-RAS Humanoids*, 2017, pp. 577–584.
- [2] R. Grandia, F. Jenelten, S. Yang, F. Farshidian, and M. Hutter, “Perceptive locomotion through nonlinear model-predictive control,” *IEEE Transactions on Robotics*, vol. 39, no. 5, pp. 3402–3421, 2023.
- [3] E. Todorov and W. Li, “A generalized iterative lqg method for locally-optimal feedback control of constrained nonlinear stochastic systems,” in *American Control Conference, 2005.*, 2005, pp. 300–306 vol. 1.
- [4] M. Gifthalder, M. Neunert, M. Stäuble, J. Buchli, and M. Diehl, “A family of iterative gauss-newton shooting methods for nonlinear optimal control,” in *IEEE/RSJ IROS*, 2018, pp. 1–9.
- [5] C. Mastalli, R. Budhiraja, W. Merkt, G. Saurel, B. Hammoud, M. Naveau, J. Carpentier, L. Righetti, S. Vijayakumar, and N. Mansard, “Crocodyl: An Efficient and Versatile Framework for Multi-Contact Optimal Control,” in *IEEE ICRA*, 2020.
- [6] E. Dantec, M. Naveau, P. Fernbach, N. Villa, G. Saurel, O. Stasse, M. Taix, and N. Mansard, “Whole-body model predictive control for biped locomotion on a torque-controlled humanoid robot,” in *IEEE-RAS Humanoids*, 2022, pp. 638–644.
- [7] F. Farshidian *et al.*, “OCS2: An open source library for optimal control of switched systems.”
- [8] V. Robin, F. Gianluca, K. Dimitris, F. Jonathan, D. Niels, Z. Andrea, N. Branimir, A. Thivaharan, Q. Rien, and D. Moritz, “acados a modular open-source framework for fast embedded optimal control,” 2022.
- [9] F. Jenelten, R. Grandia, F. Farshidian, and M. Hutter, “Tamols: Terrain-aware motion optimization for legged systems,” *IEEE Transactions on Robotics*, vol. 38, no. 6, pp. 3395–3413, 2022.
- [10] N. Rathod, A. Bratta, M. Focchi, M. Zanon, O. Villarreal, C. Semini, and A. Bemporad, “Model predictive control with environment adaptation for legged locomotion,” *IEEE Access*, vol. 9, 2021.
- [11] A. Jordana, S. Kleff, A. Meduri, J. Carpentier, N. Mansard, and L. Righetti, “Stagewise implementations of sequential quadratic programming for model predictive control,” 2023.
- [12] S. H. Jeon, S. Hong, H. J. Lee, C. Khazoom, and S. Kim, “Cusadi: A gpu parallelization framework for symbolic expressions and optimal control,” *IEEE Robotics and Automation Letters*, vol. 10, no. 2, pp. 899–906, 2025.
- [13] J. A. E. Andersson, J. Gillis, G. H. J. B. Rawlings, and M. Diehl, “CasADi: A software framework for nonlinear optimization and optimal control,” *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.
- [14] A. L. Bishop, J. Z. Zhang, S. Gurumurthy, K. Tracy, and Z. Manchester, “Relu-qg: A gpu-accelerated quadratic programming solver for model-predictive control,” in *IEEE ICRA*, 2024, pp. 13 285–13 292.
- [15] Y. Lee, M. Cho, and K.-S. Kim, “Gpu-parallelized iterative lqr with input constraints for fast collision avoidance of autonomous vehicles,” in *IEEE/RSJ IROS*, 2022, pp. 4797–4804.
- [16] Y. Lee, K. H. Choi, and K.-S. Kim, “Gpu-enabled parallel trajectory optimization framework for safe motion planning of autonomous vehicles,” *IEEE Robotics and Automation Letters*, vol. 9, no. 11, pp. 10 407–10 414, 2024.
- [17] B. Plancher and S. Kuindersma, “A performance analysis of parallel differential dynamic programming on a gpu,” in *Algorithmic Foundations of Robotics XIII*, M. Morales, L. Tapia, G. Sánchez-Ante, and S. Hutchinson, Eds. Cham: Springer International Publishing, 2020, pp. 656–672.
- [18] J. V. Frasch, S. Sager, and M. Diehl, “A parallel quadratic programming method for dynamic optimization problems,” *Mathematical Programming Computation*, 2015.
- [19] S. J. Wright, “Partitioned dynamic programming for optimal control,” *SIAM Journal on Optimization*, vol. 1, no. 4, pp. 620–642, 1991. [Online]. Available: <https://doi.org/10.1137/0801037>
- [20] S. Särkkä and A. F. García-Fernández, “Temporal parallelization of dynamic programming and linear quadratic control,” *IEEE Transactions on Automatic Control*, vol. 68, no. 2, pp. 851–866, 2023.
- [21] C. Mastalli, S. Chhatoi, T. Corbères, S. Tonneau, and S. Vijayakumar, “Inverse-dynamics mpc via nullspace resolution,” *IEEE Transactions on Robotics*, vol. 39, no. 4, pp. 3222–3241, 2023.
- [22] Z. Yoon, J.-H. Kim, and H.-W. Park, “Invariant smoother for legged robot state estimation with dynamic contact event information,” *IEEE Transactions on Robotics*, vol. 40, pp. 193–212, 2024.
- [23] S. Martinez, R. Griffin, and C. Mastalli, “Multi-contact inertial estimation and localization in legged robots,” in *IEEE/RSJ IROS*, 2024.
- [24] L. Amatucci, G. Turrissi, A. Bratta, V. Barasuol, and C. Semini, “Accelerating model predictive control for legged robots through distributed optimization,” in *IEEE/RSJ IROS*, 2024.
- [25] S. C. Flavio De Vincenti, “Centralized model predictive control for collaborative loco-manipulation,” in *RSS*, 2023.
- [26] J. Kim, R. T. Fawcett, V. R. Kamidi, A. D. Ames, and K. A. Hamed, “Layered control for cooperative locomotion of two quadrupedal robots: Centralized and distributed approaches,” *IEEE Transactions on Robotics*, vol. 39, no. 6, pp. 4728–4748, 2023.
- [27] X. Cheng, K. Shi, A. Agarwal, and D. Pathak, “Extreme parkour with legged robots,” in *CoRL*, 2023.
- [28] F. Jenelten, J. He, F. Farshidian, and M. Hutter, “Dtc: Deep tracking control,” *Science Robotics*, vol. 9, no. 86, p. eadh5401, 2024.
- [29] S. Omar, L. Amatucci, V. Barasuol, G. Turrissi, and C. Semini, “Safesteps: Learning safer footstep planning policies for legged robots via model-based priors,” in *IEEE-RAS Humanoids*, 2023, pp. 1–8.
- [30] A. Pandala, R. T. Fawcett, U. Rosolia, A. D. Ames, and K. A. Hamed, “Robust predictive control for quadrupedal locomotion: Learning to close the gap between reduced- and full-order models,” *IEEE Robotics and Automation Letters*, vol. 7, no. 3, pp. 6622–6629, 2022.
- [31] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, “JAX: composable transformations of Python+NumPy programs.”
- [32] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, A. Mandlikar, B. Babich, G. State, M. Hutter, and A. Garg, “Orbit: A unified simulation framework for interactive robot learning environments,” *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3740–3747, 2023.
- [33] E. Todorov, T. Erez, and Y. Tassa, “Mujoco: A physics engine for model-based control,” in *IEEE/RSJ IROS*. IEEE, 2012, pp. 5026–5033.
- [34] D. Kiessling, K. Baumgärtner, J. Frey, W. Decré, J. Swevers, and M. Diehl, “Fast generation of feasible trajectories in direct optimal control,” *IEEE Control Systems Letters*, vol. 8, pp. 1168–1173, 2024.
- [35] G. Blelloch, “Scans as primitive parallel operations,” *IEEE Transactions on Computers*, vol. 38, no. 11, pp. 1526–1538, 1989.
- [36] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge University Press, 2004.
- [37] L. Csanky, “Fast parallel matrix inversion algorithms,” *SIAM Journal on Computing*, vol. 5, no. 4, pp. 618–623, 1976.
- [38] A. Wächter and L. T. Biegler, “On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming,” *Mathematical Programming*, vol. 106, 2006.
- [39] L. Armijo, “Minimization of functions having lipschitz continuous first partial derivatives,” *Pacific Journal of mathematics*, vol. 16, no. 1, pp. 1–3, 1966.
- [40] S. Gros, M. Zanon, R. Quirynen, A. Bemporad, and M. Diehl, “From linear to nonlinear mpc: bridging the gap via the real-time iteration,” *International Journal of Control*, vol. 93, no. 1, pp. 62–80, 2020.
- [41] C. Khazoom, S. Hong, M. Chignoli, E. Stanger-Jones, and S. Kim, “Tailoring solution accuracy for fast whole-body model predictive control of legged robots,” *IEEE Robotics and Automation Letters*, vol. 9, no. 12, pp. 11 074–11 081, 2024.
- [42] G. Bledt, M. J. Powell, B. Katz, J. Di Carlo, P. M. Wensing, and S. Kim, “Mit cheetah 3: Design and control of a robust, dynamic quadruped robot,” in *IEEE/RSJ IROS*, 2018, pp. 2245–2252.
- [43] Y. Nisticò, J. C. V. Soares, L. Amatucci, G. Fink, and C. Semini, “Muse: A real-time multi-sensor state estimator for quadruped robots,” *IEEE Robotics and Automation Letters*, vol. 10, no. 5, pp. 4620–4627, 2025.
- [44] I. Taouil, L. Amatucci, M. Khadiv, A. Dai, V. Barasuol, G. Turrissi, and C. Semini, “Non-gaited legged locomotion with monte-carlo tree search and supervised learning,” *IEEE Robotics and Automation Letters*, vol. 10, no. 2, pp. 1265–1272, 2025.